

CSE 620 Project 1: Gradient Descent Variants

Fall 2026

John Brittan

Robert Lovett

Abstract—This report documents a wellthought-out and precise carrying out of the first project for CSE 620.

Index Terms—heuristics, gradient descent, optimization, fun

I. INTRODUCTION

The objective of this project is to implement and explore several variants of the gradient descent optimization algorithm. Each variant will be used to minimize each of the following functions. The only constraint is that these functions are all of the form $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, so we're minimizing over the entire plane.

$$f_1(x, y) = x^2 + y^2 \quad (1)$$

$$f_2(x, y) = (1 - x)^2 + 100(y - x^2)^2 \quad (2)$$

$$f_3(x, y) = x^2 + y^2 + 10 \cos x + 10 \cos y \quad (3)$$

We've implemented and experimented with:

- 1) Vanilla Gradient Descent
- 2) Newton's Method (w/ Hessian)
- 3) AdaGrad
- 4) Adam (w/ $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e - 8$)

II. METHODS

Here we describe in more detail the gradient descent algorithms used, and our implementations of them.

A. Gradient Descent Algorithms

1) Vanilla Gradient Descent

This one isn't anything crazy. This algorithm simply starts with a random initial point (here we use a fixed initial point for consistency), then iteratively updates the point using the gradient of f :

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

This algorithm in pseudocode looks like this:

- 1: Initialize $(x, y) \leftarrow (x_0, y_0)$
- 2: Initialize $\alpha \leftarrow \alpha_0$
- 3: **for** $t = 1, \dots, T$ **do**
- 4: $(x, y) \leftarrow (x, y) - \alpha \nabla f(x, y)$
- 5: **end for**
- 6: **return** (x, y)

2) Newton's Method (w/ Hessian)

Newton's Method is a second-order optimization algorithm that uses both the gradient and the Hessian matrix of f :

$$H_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

This algorithm in pseudocode looks like this:

- 1: Initialize $(x, y) \leftarrow (x_0, y_0)$
- 2: Initialize $\alpha \leftarrow \alpha_0$
- 3: **for** $t = 1, \dots, T$ **do**
- 4: $(x, y) \leftarrow (x, y) - \alpha H_f^{-1}(x, y) \nabla f(x, y)$
- 5: **end for**
- 6: **return** (x, y)

3) AdaGrad

AdaGrad is an adaptive learning rate optimization algorithm that adjusts the learning rate for each parameter based on the historical gradients. It accumulates the squared gradients and uses them to scale the learning rate. Some implementations the entire outer product of past gradients with itself, but ours uses only the diagonal of that matrix, which is much more computationally efficient.

This algorithm in pseudocode looks like this:

- 1: Initialize $(x, y) \leftarrow (x_0, y_0)$
- 2: Initialize $\alpha \leftarrow \alpha_0$
- 3: Initialize $g_{acc} \leftarrow (0, 0)$
- 4: **for** $t = 1, \dots, T$ **do**
- 5: $g \leftarrow \nabla f(x, y)$
- 6: $g_{acc} \leftarrow g_{acc} + g \odot g$
- 7: $(x, y) \leftarrow (x, y) - \alpha \cdot (g_{acc})^{\odot -\frac{1}{2}} \cdot \nabla f(x, y)$
- 8: **end for**
- 9: **return** (x, y)

where $\odot$ denotes the [Hadamard operations](#).

4) Adam

Adam is essentially an intelligent combination of AdaGrad with momentum. It tracks an exponentially decaying average of past gradients and squared gradients, and uses these to adapt the learning rate for each parameter. In this project we use the following values for Adam's unique hyperparameters:

$$\beta_1 = 0.9 \quad \beta_2 = 0.999 \quad \epsilon = 1e - 8$$

This algorithm in pseudocode looks like this:

```

1: Initialize  $(x, y) \leftarrow (x_0, y_0)$ 
2: Initialize  $\alpha \leftarrow \alpha_0$ 
3: Initialize  $m \leftarrow (0, 0)$ 
4: Initialize  $v \leftarrow (0, 0)$ 
5: for  $t = 1, \dots, T$  do
6:    $g \leftarrow \nabla f(x, y)$ 
7:    $m \leftarrow \beta_1 m + (1 - \beta_1)g$ 
8:    $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$ 
9:    $\hat{m} \leftarrow \frac{m}{1 - \beta_1^t}$ 
10:   $\hat{v} \leftarrow \frac{v}{1 - \beta_2^t}$ 
11:   $(x, y) \leftarrow (x, y) - \alpha \cdot \left( \hat{m} \oslash \left( \sqrt{\hat{v}} + \epsilon \right) \right)$ 
12: end for
13: return  $(x, y)$ 

```

where $\oslash$ denotes the [Hadamard division](#).

ACKNOWLEDGMENT

We would like to thank and acknowledge our professor Dr. Olfa Nasraoui for her guidance and support throughout this project.